

自然言語処理 実験レポート

実験日：2025 年 6 月 20 日

提出日：2025 年 6 月 26 日

組番号：212

学籍番号：22060

氏名：古城隆人

1 目的

近年ではネットワークの進化、技術の進化により大量のデータを取得することが容易になっている。大量のデータをどのように文政するかで課題となるものの一つが自然言語である。自然言語を分析することにより、人間では気づきづらい特徴の抽出を行うことが出来る。しかしながら自然言語は普段日常で人が話すものであり、自分たちが最も理解できる言語である。そのため、自然言語を情報データとして分析するときに、計算機などは自然言語を理解することができない(二進数を基とする計算しかできないため)。そこで今回は自然言語を数値化することを目指して実験を行っていき、それぞれの手法の利点や改善点を考察していく。今回は 100 件のテキストデータに対してデータ分析を行う。混乱を避けるため、ファイル 1 件をテキスト、テキストの中の本文を文章と呼ぶことにする。

2 実験内容

本実験で実際に行った内容を具体的に述べる。
使用した手法 3 つと内容、また前提となる技術を以下に示す。

2.1 形態素解析

形態素解析では、自然言語に対して単語や文節などの単位で分割し、それぞれの単語の品詞や意味を解析する手法である。これにより、自然言語を構成する要素を抽出し、テキストデータの前処理を行うことができる。目的で書いた計算機で自然言語を理解するという目的は形態素解析により、達成することが出来ると言える。なのでこれ以下の技術は計算機で扱えるようになったデータを分析する技術となる。形態素解析には MeCab というツールを使用する。このツールは辞書型のツールであり日本語の形態素解析に特化している。

2.2 Term Frequency

Term Frequency (以下本文では TF と略す) は、文書内の単語の出現頻度を計算する手法である。単純に出現する頻度が高い文章量を求めることにより、その文章が何を表しているかを見ることが出来る。しかし比較対象とするテキストがある場合には単語の出現回数で比較すると必然的に文章 No 文字の量が多いほうが有利になってしまう。そのため正規化という手法を用いて、出現する度合いで比較することにする。計算式で表すと数式 (1) のようになる。ここで、 t は単語、 d は文書を表す。count は単語 t が文書 d に出現する回数を表し、count(d) は文書 d 内の全単語の出現回数を表す。TF は単語の出現頻度を文書内の全単語の出現頻度で割ることで、度合いを求めている。

$$TF(t, d) = \frac{\text{count}(t, d)}{\text{count}(d)} \quad (1)$$

2.3 Inverse Document Frequency

Inverse Document Frequency (以下本文では IDF と略す) は、テキスト全体での単語の希少性を求めるものである。様々な分野で使われる単語は必然的に様々な文章での出現頻度が高まる。そのため、複数のテキストの中で出現する頻度が低い単語ほど特定の分野でしか使われていない単語となる。これを求めることにより、特定の分野で使われている単語を参照するだけで関連する文章の数を減らすことが出来る。IDF は通常、式 (2) のように定義される。ここで、 t は単語、 D はテキストの集合を表す。 $|D|$ はテキストの総数、 $|\{d \in D : t \in d\}|$ は単語 t を含むテキストの数を表す。IDF は、単語 t が文書集合 D においてどれだけ希少であるかを示す。 $\log$ を使用する理由として、出現回数と頻度を逆比例の形にするよりも出現頻度が高い単語のウェイトを出来るだけ小さくして低い単語のウェイトをできるだけ大きくすることが出来るからである。例として、今回のテキスト量は 100 件であるため、 $y = \log_{10}(\frac{100}{x})$ の式となるので x の値から y を求めてみると $1 \sim 10$ の間では $1 \leq y \leq 2$ になり、 $10 \sim 100$ の間では $0 < y < 1$ となる。よって出現回数が 10 以下の単語に対してそれ以降の出現頻度のウェイトが低いことがわかる。

$$\text{IDF}(t, D) = \log \left(\frac{|D|}{|\{d \in D : t \in d\}|} \right) \quad (2)$$

2.4 tf-idf

TF-IDF は、tf と idf の値から文章中での単語の重み付けをしているものである。tf と idf の値を乗算することにより、全体のテキストの中で希少性の高い単語かつ、文章中で出現頻度が高い単語を求めることが出来る。これにより tf でも求めた値から一般に使われる単語をフィルタリングしたものが tf-idf と言える。tf-idf は、式 (3) のように定義される。ここで、 t は単語、 d は文書、 D はテキストの集合を表す。TF-IDF は、単語 t の文書 d における重要度を示す。

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D) \quad (3)$$

3 実験環境

- ハードウェア：Windows 11, Intel Core i7-11800H, 16GB RAM
- ソフトウェア：java 21.0.1 2023-10-17 LTS, MeCab 0.996, Visual Studio Code
- 使用データ：用意されたデータファイル群 (テキスト 100 件)

4 設計

4.1 プログラム設計方針

プログラムは Java で実装を行った。Java を使用した理由として、tf や idf をそれぞれオブジェクトとして扱うことが出来、プログラムの行数が少なくなり可読性があるためである。また、各オブジェクトの設計を適切に行うことにより、プログラムの実装が容易になる。そのため自然言語処理を行うための手段として最適な言語である。プログラムの設計を以下に示す。

4.1.1 Word オブジェクト

Word オブジェクトは単語を格納するオブジェクトである。形態素解析によって得たデータを Word オブジェクトに格納し、一単語ずつの特徴を扱うことが出来る。Word オブジェクトの定義である Word.class を Listing1 に示す。equals メソッドを用意することにより、Word オブジェクト同士の比較を行うことが出来る。また、toString メソッドを用意することによりクラスの中身を容易に見ることが出来、デバッグの速度をあげることが出来る。

Listing 1: Word オブジェクトの定義

```
1 package nlp;
2 class Word {
3     private String hyousoukei = null; // 表層形
4     private String hinshi = null; // 品詞
5     private String hinshi1 = null; // 品詞細分類 1
6     private String hinshi2 = null; // 品詞細分類 2
7     private String hinshi3 = null; // 品詞細分類 3
8     private String katsuyoKata = null; // 活用型
9     private String katsuyoKei = null; // 活用形
10    private String genkei = null; // 原形
11    private String yomi = null; // 読み
12    private String hatsuon = null; // 発音
13
14    public boolean equals(Object obj) {
15        // オブジェクトが null の場合、不一致
16        if (obj == null) {
17            return false;
18        }
19        // オブジェクトの型が異なる場合、不一致
20        if (!(obj instanceof Word)) {
21            return false;
22        }
23        // オブジェクトの全てのフィールドが一致している場合、一致
24        if (((Word) obj).getHyouSoukei() == null && this.getHyouSoukei() == null)
```

```

25         || ((Word) obj).getHyouSoukei().equals(this.getHyouSoukei()))
26         && (((Word) obj).getHinshi() == null && this.getHinshi() == null)
27         || ((Word) obj).getHinshi().equals(this.getHinshi()))
28         && (((Word) obj).getHinshi1() == null && this.getHinshi1() == null)
29         || ((Word) obj).getHinshi1().equals(this.getHinshi1()))
30         && (((Word) obj).getHinshi2() == null && this.getHinshi2() == null)
31         || ((Word) obj).getHinshi2().equals(this.getHinshi2()))
32         && (((Word) obj).getHinshi3() == null && this.getHinshi3() == null)
33         || ((Word) obj).getHinshi3().equals(this.getHinshi3()))
34         && (((Word) obj).getKatsuyoKata() == null && this.getKatsuyoKata()
35             == null)
36             || ((Word) obj).getKatsuyoKata().equals(this.getKatsuyoKata
37                 ()))
38         && (((Word) obj).getKatsuyoKei() == null && this.getKatsuyoKei() ==
39             null)
40             || ((Word) obj).getKatsuyoKei().equals(this.getKatsuyoKei()))
41         && (((Word) obj).getGenkei() == null && this.getGenkei() == null)
42         || ((Word) obj).getGenkei().equals(this.getGenkei()))
43         && (((Word) obj).getYomi() == null && this.getYomi() == null)
44         || ((Word) obj).getYomi().equals(this.getYomi()))
45         && (((Word) obj).getHatsuon() == null && this.getHatsuon() == null)
46         || ((Word) obj).getHatsuon().equals(this.getHatsuon())) {
47             return true;
48         }
49         // そのほかは不一致
50         return false;
51     }
52
53     public String toString() {
54         return "Word [hyousoukei=" + hyousoukei + ", hinshi=" + hinshi + ", hinshi1="
55             + hinshi1 + ", hinshi2=" + hinshi2
56             + ", hinshi3=" + hinshi3 + ", katsuyoKata=" + katsuyoKata + ",
57             katsuyoKei=" + katsuyoKei + ", genkei="
58             + genkei + ", yomi=" + yomi + ", hatsuon=" + hatsuon + "];"
59     }
60
61     // 各種ゲッターとセッター
62 }

```

4.1.2 WordCount オブジェクト

WordCount オブジェクトは、Word オブジェクトを格納し、その出現回数をカウントするオブジェクトである。単語一つにつきひとつのインスタンスを生成し、その単語の出現回数をカウントしている。WordCount オブジェクトの定義である WordCount.class を Listing2 に示す。

Listing 2: WordCount オブジェクトの定義

```
1 package nlp;
2
3 class WordCount {
4     private Word word; // 形態素解析した語の結果を格納する
5     private Integer count; // 出現回数を計数するためのカウンタ
6
7     WordCount(Word word, Integer count) {
8         this.word = word;
9         this.count = count;
10    }
11
12    // 各種ゲッターとセッター
13 }
```

4.1.3 TF オブジェクト

TF オブジェクトでは、TF 値の格納と、TF 値の導出を行う。TF 値の計算を行うために文章内で出現した単語を記憶しておく配列を用意する。Java では ArrayList を使用することにより、配列の長さが可変長の配列を扱うことが出来る。また、データの型は任意のオブジェクトにすることが出来る。そのため今回は WordCount オブジェクトを格納する ArrayList を用意する。しかし、単語の数だけインスタンスの生成を行うためメモリの使用量が増加するというデメリットがあるが、使用する計算機のメモリが大きいのと解析するファイルの量が少ないため、デメリットはほぼないに等しくなる。WordCount オブジェクトには単語と出現回数の記録能力しかないため、実際には ArrayList の DfCount オブジェクトにした。TfCount オブジェクトは WordCount オブジェクトを継承しており、TF 値を格納するための変数を追加している。TF オブジェクトの定義である TermFrequency.class を Listing3 に示す。計算結果を格納しておく配列以外にも、TF 値を求めるための関数 tf が定義されている。tf 関数では、MeCab を使用して形態素解析を行い、その結果を list に格納している。また、list に格納した結果を出力するためのファイル名を引数として受け取り、そのファイルに出力する機能も持っている。

Listing 3: TF オブジェクトの定義

```
1 package nlp;
2
3 import java.io.BufferedReader;
4 import java.io.File;
5 import java.io.IOException;
6 import java.io.InputStreamReader;
7 import java.util.ArrayList;
8
9 public class TermFrequency {
10     ArrayList<TfCount> list = new ArrayList<TfCount>();
```

```

11
12 public void tf(String inputFilename, String outputFilename) {
13     // 解析対象のファイル名
14     // String inputFilename = "data\\001.txt";
15     // mecab コマンドで対象ファイルを解析するコマンド文
16     String[] command = { "cmd.exe", "/C", "mecab", inputFilename };
17     int sum = 0;
18     try {
19         // 解析するコマンドを実行する
20         Process ps = Runtime.getRuntime().exec(command);
21         // 解析した結果を表示するためのオブジェクトに変換する
22         BufferedReader bReader_i = new BufferedReader(new InputStreamReader(ps.
            getInputStream(), "SJIS"));
23         // 標準出力を 1 行ずつ受け取る一時オブジェクト
24         String targetLine;
25         while (true) {
26             // 形態素解析結果を 1 行ずつ受け取る
27             targetLine = bReader_i.readLine();
28             if (targetLine == null) {
29                 // 最終行になったら終わる
30                 break;
31             } else if (targetLine.equals("EOS")) {
32                 continue;
33             } else {
34                 // 行ごとに結果を表示する
35                 String targetArray[] = targetLine.split("[,\\t]");
36                 Word wo = new Word();
37                 if (targetArray.length > 0) {
38                     wo.setHyouyoukei(targetArray[0]);
39                 }
40                 if (targetArray.length > 1) {
41                     wo.setHinshi(targetArray[1]);
42                 }
43                 if (targetArray.length > 2) {
44                     wo.setHinshi1(targetArray[2]);
45                 }
46                 if (targetArray.length > 3) {
47                     wo.setHinshi2(targetArray[3]);
48                 }
49                 if (targetArray.length > 4) {
50                     wo.setHinshi3(targetArray[4]);
51                 }
52                 if (targetArray.length > 5) {
53                     wo.setKatsuyoKata(targetArray[5]);
54                 }

```

```

55         if (targetArray.length > 6) {
56             wo.setKatsuyoKei(targetArray[6]);
57         }
58         if (targetArray.length > 7) {
59             wo.setGenkei(targetArray[7]);
60         }
61         if (targetArray.length > 8) {
62             wo.setYomi(targetArray[8]);
63         }
64
65         if ("名詞".equals(wo.getHinshi().toString())
66             && ("一般".equals(wo.getHinshi1().toString()) || "サ変接
67                 続".equals(wo.getHinshi1().toString())
68                 || "固有名詞".equals(wo.getHinshi1().toString()))
69             && (wo.getHyouSoukei().length() > 2)) {
70             // System.out.println(wo.toString());
71
72             int i;
73             for (i = 0; i < list.size(); i++) {
74                 if (list.get(i).getWord().equals(wo)) {
75                     list.get(i).setCount(list.get(i).getCount() + 1);
76                     break;
77                 }
78             }
79             // リストにエントリが無かったときは、新しい語としてリストに追
80             // 加する
81             if (i == list.size()) {
82                 list.add(new TfCount(wo, Integer.valueOf(1)));
83             }
84
85             // 形態素解析結果を表示する
86             // System.out.println(wo.toString());
87
88         }
89
90     }
91     for (int i = 0; i < list.size(); i++) {
92         // System.out.println(list.get(i).getWord().getHyouSoukei()
93         // + ":" + list.get(i).getCount());
94         sum += list.get(i).getCount();
95     }
96     // System.out.println("合計語数:" + sum);
97

```



```

98         list.sort(new WordCompare());
99
100         // String outputFilename = "data\\001tf.txt";
101         try (FileWriter fw = new FileWriter(outputFilename)) {
102             // System.out.println(outputFilename);
103             fw.write("単語\t出現回数\tTF\n");
104             for (int i = 0; i < list.size(); i++) {
105                 // System.out.println(list.get(i).getWord().getHyouSoukei() + ":"
106                 // + list.get(i).getCount());
107                 list.get(i).setTf((double) list.get(i).getCount() / (double) sum
108                     );
109                 fw.write(list.get(i).getWord().getHyouSoukei() + "\t"
110                     + list.get(i).getCount() + "\t"
111                     + String.format("%.10f", list.get(i).getTf()) + "\n");
112             }
113             fw.close();
114         } catch (IOException ex) {
115             ex.printStackTrace();
116         } catch (IOException e) {
117             e.printStackTrace();
118         }
119     }
120 }

```

4.1.4 IDF オブジェクト

IDF オブジェクトでは、IDF 値の格納と、IDF 値の導出を行う。TF オブジェクト同様に ArrayList を使用して、IDF 値を求めるための配列を用意する。IDF の計算に使用する数式は実験内容で示したものと違い、数式 (2) を使用して求める。IDF オブジェクトの定義である DocumentFrequency.class を Listing4 に示す。IDF オブジェクトでは、tf-idf の計算を行うために TF オブジェクトを引数として受け取り、その TF オブジェクトの中に格納されている WordCount オブジェクトを用いて IDF 値を求める関数 idf を定義している。しかし、TF オブジェクトと同様に DfCount オブジェクトを新たに作成して DfCount オブジェクトの配列を用いている。

$$\text{IDF}(t, D) = \log \left(\frac{|D|}{|\{d \in D : t \in d\}|} \right) \quad (4)$$

Listing 4: IDF オブジェクトの定義

```

1     package nlp;
2
3     import java.io.FileWriter;
4     import java.io.IOException;

```

```

5  import java.util.ArrayList;
6
7  public class DocumentFrequency {
8      // DF をカウントするためのデータ格納領域の定義
9      ArrayList<DfCount> dfList = new ArrayList<DfCount>();
10
11     // DF の元になる TF を受ける
12     TermFrequency tf[];
13
14     DocumentFrequency(TermFrequency[] tf) {
15         this.tf = tf;
16     }
17
18     public void df(String outputFilename) {
19         // TF100 ファイル分について繰り返す
20         for (int i = 0; i < tf.length; i++) {
21
22             // TF1 件に含まれる語の分だけ繰り返し
23             for (TfCount tfCount : tf[i].list) {
24                 // DF のリストの中にエントリがあるか調べる
25                 for (int j = 0; j < dfList.size(); j++) {
26                     // エントリがあった場合はカウントを増やす
27                     if (dfList.get(j).getWord().equals(tfCount.getWord())) {
28                         dfList.get(j).setCount(dfList.get(j).getCount() + 1);
29                         break;
30                     }
31                 }
32                 // リストにエントリが無かったときは、新しい語としてリストに追加する
33                 DfCount df = new DfCount(tfCount.getWord(), 1);
34                 dfList.add(df);
35
36             }
37
38         }
39         // ソートする
40         dfList.sort(new DfCompare());
41         // df, idf の結果をファイルに保存する
42
43         try (FileWriter fw = new FileWriter(outputFilename)) {
44             fw.write("単語\tDF\tIDF\n");
45             for (int j = 0; j < dfList.size(); j++) {
46                 dfList.get(j)
47                     .setIdf(Math.log10((double) tf.length / (double) dfList.get(j)
48                                     .getCount()) + 1.0);
49                 if (dfList.get(j).getIdf() < 1.005) {

```

```

49         System.out.println(dfList.get(j).getWord().getHyouSoukei() + "\t"
50                               + dfList.get(j).getCount()
51                               + "\t" + String.format("%.10f", dfList.get(j).getIdf()));
52     } else {
53         fw.write(dfList.get(j).getWord().getHyouSoukei() + "\t" + dfList.
54                   get(j).getCount() + "\t"
55                   + String.format("%.10f", dfList.get(j).getIdf()) + "\n");
56     }
57     fw.close();
58 } catch (IOException ex) {
59     ex.printStackTrace();
60 }
61
62 }
63
64 }

```

4.1.5 TF-IDF オブジェクト

TF-IDF オブジェクトでは、tf-idf 値の格納と、tf-idf 値の導出を行う。TF オブジェクト同様に ArrayList を使用して、tf-idf 値を求めるための配列を用意する。TF-IDF オブジェクトの定義である tfidf.class を Listing5 に示す。TF-IDF オブジェクトでは、IDF オブジェクトを引数として受け取り、その IDF オブジェクトの中に格納されている DfCount オブジェクト、TfCount オブジェクトを用いて tf-idf 値を求める関数 tfidf を定義している。

Listing 5: TF-IDF オブジェクトの定義

```

1     package nlp;
2
3     import java.io.FileWriter;
4     import java.io.IOException;
5     import java.util.ArrayList;
6
7     public class TfIdf {
8         // TF-IDF を計算するためのデータ格納領域の定義
9         ArrayList<TfIdfCount> tfIdfList = new ArrayList<TfIdfCount>();
10
11         // TF-IDF の元になる DF を受ける
12         DocumentFrequency df;
13
14         TfIdf(DocumentFrequency df) {
15             this.df = df;
16         }

```

```

17
18 public void tfidf(String outputFilename) {
19     int j = 0;
20     for (TermFrequency tf : df.tf) {
21         for (TfCount tfCount : tf.list) {
22             for (DfCount dfCount : df.dfList) {
23                 // TF の単語が DF の単語と一致するか確認
24                 if (tfCount.getWord().equals(dfCount.getWord())) {
25                     if (tfCount.getTf() * dfCount.getIdf() > 0.011) { // TF-IDF
26                         // の値が 0.1 より大きい場合のみ追加
27                         TfidfCount newTfidfCount = new TfidfCount(tfCount.getWord
28                             (), tfCount.getCount(),
29                             tfCount.getTf(),
30                             dfCount.getIdf(), tfCount.getTf() * dfCount.
31                                 getIdf());
32                         tfIdfList.add(newTfidfCount);
33                     }
34                     break;
35                 }
36             }
37         }
38     }
39
40     // ソートする
41     tfIdfList.sort(new TfidfCompare());
42
43     outputFilename = "data\\" + String.format("%03d", j + 1) + "tfidf.txt";
44     // 出力ファイル名を設定
45     // tf-idf の結果をファイルに保存する
46     try (FileWriter fw = new FileWriter(outputFilename)) {
47         fw.write("単語\t出現回数\tTF\tIDF\ttfidf\n");
48         for (TfidfCount tfidfCount : tfIdfList) {
49             // 単語 出現回数 TF IDF tfidf
50             fw.write(tfidfCount.getWord().getHyouSoukei() + "\t"
51                 + tfidfCount.getCount() + "\t"
52                 + String.format("%.10f", tfidfCount.getTf()) + "\t"
53                 + String.format("%.10f", tfidfCount.getIdf()) + "\t"
54                 + String.format("%.10f", tfidfCount.getTfIdf()) + "\n");
55         }
56         fw.close();
57     } catch (IOException ex) {
58         ex.printStackTrace();
59     }
60     tfIdfList.clear(); // 次のTF-IDF計算のためにリストをクリア
61     j++; // 次のファイルのインデックスに進む
62 }

```

```
58     }  
59  
60 }
```

4.1.6 main 関数

main 関数では、実際にプログラムを実行するための関数である。main 関数では、実験内容で示した手法を順番に実行していく。NaturalLanguageProcessing.class を Listing6 に示す。

Listing 6: main 関数の定義

```
1     package nlp;  
2  
3     public class NaturalLanguageProcessing {  
4         static public void main(String args[]) {  
5             System.out.println("TF 導出");  
6             TermFrequency[] tf = new TermFrequency[100];  
7             for (int i = 1; i <= 100; i++) {  
8                 tf[i - 1] = new TermFrequency();  
9                 String inputFileName = "data\\" + String.format("%03d", i) + ".txt";  
10                String outputFileName = "data\\" + String.format("%03d", i) + "tf.txt";  
11                tf[i - 1].tf(inputFileName, outputFileName);  
12            }  
13            System.out.println("TF 導出完了");  
14            System.out.println("DF 導出");  
15            DocumentFrequency df = new DocumentFrequency(tf);  
16            df.df("data\\df.txt");  
17            System.out.println("DF 導出完了");  
18  
19            System.out.println("TF-IDF 導出");  
20            TfIdf tfidf = new TfIdf(df);  
21            tfidf.tfidf("data\\tfidf.txt");  
22            System.out.println("TF-IDF 導出完了");  
23            System.out.println("自然言語処理完了");  
24        }  
25    }
```

5 結果

今回の実験では、言葉の種類によりフィルターをかけ、また出現頻度でのフィルターも実装し、最終的な結果での小さすぎる値のフィルターを実装した。これら 3 つのフィルターをまとめて改善案とする。各手法の改善案の結果と改善前の結果を示す。

5.1 TF 値

改善前の TF 値の計算は表 1 に示す結果となった。改善後の TF 値の計算結果を表 2 に示す。

表 1: TF 値の計算結果

単語	出現回数	TF 値
%	41	0.0764925373
天然記念物	16	0.0298507463
E	16	0.0298507463
の	13	0.0242537313
地	13	0.0242537313
〔	11	0.0205223881
県	11	0.0205223881

表 2: 改善後の TF 値の計算結果

単語	出現回数	TF 値
天然記念物	16	0.2025316456
両生類	7	0.0886075949
爬虫類	7	0.0886075949
オオサンショウウオ	5	0.0632911392
文化財	3	0.0379746835

5.2 IDF 値

改善前の IDF 値の計算は表 3 に示す結果となった。改善後の IDF 値の計算結果を表 4 に示す。

表 3: IDF 値の計算結果

単語	DF 値	IDF 値
出展	100	1.0000000000
移動	100	1.0000000000
http	100	1.0000000000
中略		
」	1	3.0000000000
作成	1	3.0000000000
カテゴリ	1	3.0000000000

表 4: 改善後の IDF 値の計算結果

単語	DF 値	IDF 値
リンク	52	1.2839966564
アメリカ	19	1.7212463990
所在地	15	1.8239087409
中略		
交響楽	1	3.0000000000
ハリウッド	1	3.0000000000
マザーグース	1	3.0000000000

5.3 TF-IDF 値

改善前の TF-IDF 値の計算は表 5 に示す結果となった。改善後の TF-IDF 値の計算結果を表 6 に示す。

表 5: TF-IDF 値の計算結果

単語	出現回数	TF	IDF	tfidf
天然記念物	16	0.0298507463	2.6989700043	0.0805662688
%	41	0.0764925373	1.0132282657	0.0775044009
{	11	0.0205223881	3.0000000000	0.0615671642
}	11	0.0205223881	3.0000000000	0.0615671642

表 6: 改善後の TF-IDF 値の計算結果

単語	出現回数	TF	IDF	tfidf
天然記念物	16	0.2025316456	2.6989700043	0.5466268363
両生類	7	0.0886075949	3.0000000000	0.2658227848
爬虫類	7	0.0886075949	2.6989700043	0.2391492409
オオサンショウウオ	5	0.0632911392	3.0000000000	0.1898734177

5.4 分析結果

ファイル番号 = $\{(\text{学籍番号}) \cdot (\text{組番号}) + 24\} \% 100 + 1$ としたファイルの分析結果を表 7 に示す。自分は学籍番号 22060, 組番号 212 のため, ファイル番号は 45 となる。

表 7: 分析結果

単語	出現回数	TF	IDF	tfidf
グリーン	13	0.3095238095	2.5228787453	0.7808910402
新幹線	2	0.0476190476	3.0000000000	0.1428571429
シート	2	0.0476190476	2.6989700043	0.1285223812
レベル	2	0.0476190476	2.5228787453	0.1201370831
東海道新幹線	1	0.0238095238	3.0000000000	0.0714285714
こだま	1	0.0238095238	3.0000000000	0.0714285714
申し出	1	0.0238095238	3.0000000000	0.0714285714
払い戻し	1	0.0238095238	3.0000000000	0.0714285714
南海電気鉄道	1	0.0238095238	3.0000000000	0.0714285714
サザン	1	0.0238095238	3.0000000000	0.0714285714
リクライニング	1	0.0238095238	3.0000000000	0.0714285714
ロングシート	1	0.0238095238	3.0000000000	0.0714285714
クロス	1	0.0238095238	2.6989700043	0.0642611906
東日本旅客鉄道	1	0.0238095238	2.6989700043	0.0642611906
名古屋	1	0.0238095238	2.3979400087	0.0570938097
サービス	1	0.0238095238	2.3010299957	0.0547864285
東日本	1	0.0238095238	2.3010299957	0.0547864285
http	1	0.0238095238	1.0043648054	0.0239134477
://	1	0.0238095238	1.0043648054	0.0239134477
wikipedia	1	0.0238095238	1.0043648054	0.0239134477
org	1	0.0238095238	1.0043648054	0.0239134477
wiki	1	0.0238095238	1.0043648054	0.0239134477
カテゴリ	1	0.0238095238	1.0043648054	0.0239134477
フリー	1	0.0238095238	1.0000000000	0.0238095238
ウィキペディア	1	0.0238095238	1.0000000000	0.0238095238
Wikipedia	1	0.0238095238	1.0000000000	0.0238095238
ナビゲーション	1	0.0238095238	1.0000000000	0.0238095238

6 考察

実験結果から読み取ったこととなぜこのような改善をしたか、また期待した値が取得できたかについて示す。

6.1 TF 値

TF 値の計算では精度の向上と処理速度アップのために登録する単語を少なうするフィルターを追加した。これにより文章の特徴を一番表している名詞の中でも重要な名詞だけを抽出することが出来ると考えた。実際の実験結果では県や地など、特定につながらない単語が上位を占めていたが、改善後には固有名詞が大半になり、TF 値の上位を見るだけで文章の特徴を把握できるようになったのでこのフィルターの追加は成功といえる。また、処理速度は 20381ms から 4893ms となった。この秒数は全体の処理時間であるが、TF のフィルターのみを有効化しただけでの改善速度である。そのためこの改善案は成功といえる。

6.2 IDF 値

IDF 値でも TF 値と同様にフィルターを追加した。このフィルターでは一般的すぎる単語の排除を目的としている。この単語は文章の特徴を表すものではないため、IDF 値の計算においては必要ないと考えた。例えば改善前には出展や移動など特徴が少ない単語がメモリ上に格納されていた。しかしフィルターを追加した後にはこのような単語が消えて固有名詞に近いものが下位を示している。そのためメモリの使用料や処理速度を改善することが出来ている。このフィルターのみを有効化した場合の処理速度は 8859ms から 8982ms であり、あまり変化はなかった。そのためこのフィルターは意味がないといえる。

6.3 TF-IDF 値

TF-IDF 値の計算では、TF-IDF の値が一定以下のものは実行結果に影響を与えないようにした。処理速度を速くしようとしたときにファイルの書き込みは大変時間がかかるものである。そのため、TF-IDF 値が 0.01 以下のものは出力しないようにした。改善前と改善後では TF-IDF 値の計算の速度が 494ms から 450ms となった、+10% の改善となった。そのためこの改善案は成功といえる。

6.4 分析結果

分析結果より、上位の単語を見るとグリーン、新幹線とあることから新幹線の座席の種類について言っていることがわかる。しかし、この文章のファイルは自由席を表しているものである。なぜ自由席という単語がないかを考えると、Mecab での形態素解析で自由と席の二つの単語分かれて

しまっているからだと考察できる。実際に Mecab で追実験を行った。実行結果を Listing7 に示す。このような単語を自然言語処理で拾うためには形態素解析のツールの改善が必要である。辞書型の形態素解析ではこのような限界があるため他の解析ツールを用いてから自然言語処理を行うことによって精度の向上が期待できる。

Listing 7: Mecab での形態素解析結果

```
1 C:\Users\kojor_fyn5sjo>mecab
2 自由席
3 自由   名詞,形容動詞語幹,*,*,*,*,自由,ジユウ,ジュー
4 席     名詞,一般,*,*,*,*,席,セキ,セキ
5 EOS
```

7 付録

付録として今回の実験で使ったソースコードを示す。

DfCompare.java

Listing 8: DfCompare.java

```
1 package nlp;
2
3 public class DfCompare implements java.util.Comparator<DfCount> {
4     @Override
5     public int compare(DfCount wc1, DfCount wc2) {
6         if (wc1.getCount() < wc2.getCount())
7             return 1;
8         if (wc1.getCount() == wc2.getCount())
9             return 0;
10        if (wc1.getCount() > wc2.getCount())
11            return -1;
12        return 0;
13    }
14
15 }
```

DfCount.java

Listing 9: DfCount.java

```
1 package nlp;
2
```

```

3  class DfCount extends WordCount {
4      private Double idf;
5
6      DfCount(Word word, Integer count) {
7          super(word, count);
8          this.idf = 0.0;
9      }
10
11     DfCount(Word word, Integer count, Double idf) {
12         super(word, count);
13         this.idf = idf;
14     }
15
16     void setIdf(Double idf) {
17         this.idf = idf;
18     }
19
20     Double getIdf() {
21         return idf;
22     }
23 }

```

DocumentFrequency.java

Listing 10: DocumentFrequency.java

```

1  package nlp;
2
3  import java.io.FileWriter;
4  import java.io.IOException;
5  import java.util.ArrayList;
6
7  public class DocumentFrequency {
8      // DF をカウントするためのデータ格納領域の定義
9      ArrayList<DfCount> dfList = new ArrayList<DfCount>();
10
11     // DF の元になる TF を受ける
12     TermFrequency tf[];
13
14     DocumentFrequency(TermFrequency[] tf) {
15         this.tf = tf;
16     }
17
18     public void df(String outputFilename) {

```

```

19 // TF100 ファイル分について繰り返す
20 for (int i = 0; i < tf.length; i++) {
21
22     // TF1 件に含まれる語の分だけ繰り返し
23     for (TfCount tfCount : tf[i].list) {
24         // DF のリストの中にエントリがあるか調べる
25         for (int j = 0; j < dfList.size(); j++) {
26             // エントリがあった場合はカウントを増やす
27             if (dfList.get(j).getWord().equals(tfCount.getWord())) {
28                 dfList.get(j).setCount(dfList.get(j).getCount() + 1);
29                 break;
30             }
31         }
32         // リストにエントリが無かったときは、新しい語としてリストに追加する
33         DfCount df = new DfCount(tfCount.getWord(), 1);
34         dfList.add(df);
35
36     }
37
38 }
39 // ソートする
40 dfList.sort(new DfCompare());
41 // df,idf の結果をファイルに保存する
42
43 try (FileWriter fw = new FileWriter(outputFilename)) {
44     fw.write("単語\tDF\tIDF\n");
45     for (int j = 0; j < dfList.size(); j++) {
46         dfList.get(j)
47             .setIdf(Math.log10((double) tf.length / (double) dfList.get(j)
48                 .getCount()) + 1.0);
49         if (dfList.get(j).getIdf() < 1.005) {
50             System.out.println(dfList.get(j).getWord().getHyouSoukei() + "\t"
51                 + dfList.get(j).getCount()
52                 + "\t" + String.format("%.10f", dfList.get(j).getIdf()));
53         } else {
54             fw.write(dfList.get(j).getWord().getHyouSoukei() + "\t" + dfList.
55                 get(j).getCount() + "\t"
56                 + String.format("%.10f", dfList.get(j).getIdf()) + "\n");
57         }
58     }
59     fw.close();
60 } catch (IOException ex) {
61     ex.printStackTrace();
62 }

```

```
61
62     }
63
64 }
```

NaturalLanguageProcessing.java

Listing 11: NaturalLanguageProcessing.java

```
1  package nlp;
2
3  public class NaturalLanguageProcessing {
4      static public void main(String args[]) {
5          System.out.println("TF 導出");
6          TermFrequency[] tf = new TermFrequency[100];
7          for (int i = 1; i <= 100; i++) {
8              tf[i - 1] = new TermFrequency();
9              String inputFileName = "data\\" + String.format("%03d", i) + ".txt";
10             String outputFileName = "data\\" + String.format("%03d", i) + "tf.txt";
11             tf[i - 1].tf(inputFileName, outputFileName);
12         }
13         System.out.println("TF 導出完了");
14         System.out.println("DF 導出");
15         DocumentFrequency df = new DocumentFrequency(tf);
16         df.df("data\\df.txt");
17         System.out.println("DF 導出完了");
18
19         long start = System.currentTimeMillis(); // 開始時刻を記録
20         System.out.println("TF-IDF 導出");
21         TfIdf tfidf = new TfIdf(df);
22         tfidf.tfidf("data\\tfidf.txt");
23         System.out.println("TF-IDF 導出完了");
24         System.out.println("自然言語処理完了");
25         long end = System.currentTimeMillis(); // 終了時刻を記録
26         System.out.println("TF導出時間: " + (end - start) + " ms"); // 経過時間を表示
27     }
28 }
```

TermFrequency.java

Listing 12: TermFrequency.java

```
1  package nlp;
```

```

2
3 import java.io.BufferedReader;
4 import java.io.FileWriter;
5 import java.io.IOException;
6 import java.io.InputStreamReader;
7 import java.util.ArrayList;
8
9 public class TermFrequency {
10     ArrayList<TfCount> list = new ArrayList<TfCount>();
11
12     // public static void main(String[] args) {
13     public void tf(String inputFilename, String outputFilename) {
14         // 解析対象のファイル名
15         // String inputFilename = "data\\001.txt";
16         // mecab コマンドで対象ファイルを解析するコマンド文
17         String[] command = { "cmd.exe", "/C", "mecab", inputFilename };
18         int sum = 0;
19         try {
20             // 解析するコマンドを実行する
21             Process ps = Runtime.getRuntime().exec(command);
22             // 解析した結果を表示するためのオブジェクトに変換する
23             BufferedReader bReader_i = new BufferedReader(new InputStreamReader(ps.
                getInputStream(), "SJIS"));
24             // 標準出力を 1 行ずつ受け取る一時オブジェクト
25             String targetLine;
26             while (true) {
27                 // 形態素解析結果を 1 行ずつ受け取る
28                 targetLine = bReader_i.readLine();
29                 if (targetLine == null) {
30                     // 最終行になったら終わる
31                     break;
32                 } else if (targetLine.equals("EOS")) {
33                     continue;
34                 } else {
35                     // 行ごとに結果を表示する
36                     String targetArray[] = targetLine.split("[,\\t]");
37                     Word wo = new Word();
38                     if (targetArray.length > 0) {
39                         wo.setHyouyoukei(targetArray[0]);
40                     }
41                     if (targetArray.length > 1) {
42                         wo.setHinshi(targetArray[1]);
43                     }
44                     if (targetArray.length > 2) {
45                         wo.setHinshi1(targetArray[2]);

```

```

46     }
47     if (targetArray.length > 3) {
48         wo.setHinshi2(targetArray[3]);
49     }
50     if (targetArray.length > 4) {
51         wo.setHinshi3(targetArray[4]);
52     }
53     if (targetArray.length > 5) {
54         wo.setKatsuyoKata(targetArray[5]);
55     }
56     if (targetArray.length > 6) {
57         wo.setKatsuyoKei(targetArray[6]);
58     }
59     if (targetArray.length > 7) {
60         wo.setGenkei(targetArray[7]);
61     }
62     if (targetArray.length > 8) {
63         wo.setYomi(targetArray[8]);
64     }
65
66     if ("名詞".equals(wo.getHinshi().toString())
67         && ("一般".equals(wo.getHinshi1().toString()) || "サ変接
68             続".equals(wo.getHinshi1().toString())
69             || "固有名詞".equals(wo.getHinshi1().toString()))
70         && (wo.getHyousoukei().length() > 2)) {
71         // System.out.println(wo.toString());
72
73         int i;
74         for (i = 0; i < list.size(); i++) {
75             if (list.get(i).getWord().equals(wo)) {
76                 list.get(i).setCount(list.get(i).getCount() + 1);
77                 break;
78             }
79         }
80         // リストにエントリが無かったときは、新しい語としてリストに追
81         // 加する
82         if (i == list.size()) {
83             list.add(new TfCount(wo, Integer.valueOf(1)));
84         }
85     }
86
87     // 形態素解析結果を表示する
88     // System.out.println(wo.toString());

```



```

89         }
90
91     }
92     for (int i = 0; i < list.size(); i++) {
93         // System.out.println(list.get(i).getWord().getHyouSoukei()
94         // + ":" + list.get(i).getCount());
95         sum += list.get(i).getCount();
96     }
97     // System.out.println("合計語数:" + sum);
98
99     list.sort(new WordCompare());
100
101     // String outputFilename = "data\\001tf.txt";
102     try (FileWriter fw = new FileWriter(outputFilename)) {
103         // System.out.println(outputFilename);
104         fw.write("単語\t出現回数\tTF\n");
105         for (int i = 0; i < list.size(); i++) {
106             // System.out.println(list.get(i).getWord().getHyouSoukei() + ":"
107             // + list.get(i).getCount());
108             list.get(i).setTf((double) list.get(i).getCount() / (double) sum
109             );
109             fw.write(list.get(i).getWord().getHyouSoukei() + "\t"
110             + list.get(i).getCount() + "\t"
111             + String.format("%.10f", list.get(i).getTf()) + "\n");
112         }
113         fw.close();
114     } catch (IOException ex) {
115         ex.printStackTrace();
116     }
117 } catch (IOException e) {
118     e.printStackTrace();
119 }
120 }
121 }

```

TfCount.java

Listing 13: TfCount.java

```

1 package nlp;
2
3 class TfCount extends WordCount {
4     private Double tf; // 計数値を全語数で割った TF 値
5

```

```

6     TfCount(Word word, Integer count) {
7         super(word, count);
8     }
9
10    TfCount(Word word, Integer count, Double tf) {
11        super(word, count);
12        this.tf = tf;
13    }
14
15    void setTf(Double tf) {
16        this.tf = tf;
17    }
18
19    Double getTf() {
20        return tf;
21    }
22 }

```

TfIdf.java

Listing 14: TfIdf.java

```

1  package nlp;
2
3  import java.io.FileWriter;
4  import java.io.IOException;
5  import java.util.ArrayList;
6
7  public class TfIdf {
8      // TF-IDF を計算するためのデータ格納領域の定義
9      ArrayList<TfIdfCount> tfIdfList = new ArrayList<TfIdfCount>();
10
11     // TF-IDF の元になる DF を受ける
12     DocumentFrequency df;
13
14     TfIdf(DocumentFrequency df) {
15         this.df = df;
16     }
17
18     public void tfidf(String outputFilename) {
19         int j = 0;
20         for (TermFrequency tf : df.tf) {
21             for (TfCount tfCount : tf.list) {
22                 for (DfCount dfCount : df.dfList) {

```

```

23         // TF の単語が DF の単語と一致するか確認
24         if (tfCount.getWord().equals(dfCount.getWord())) {
25             if (tfCount.getTf() * dfCount.getIdf() > 0.011) { // TF-IDF
                の値が 0.1 より大きい場合のみ追加
26                 TfidfCount newTfidfCount = new TfidfCount(tfCount.getWord
                    (), tfCount.getCount(),
27                     tfCount.getTf(),
28                     dfCount.getIdf(), tfCount.getTf() * dfCount.
                        getIdf());
29                 tfidfList.add(newTfidfCount);
30             }
31             break;
32         }
33     }
34 }
35
36 // ソートする
37 tfidfList.sort(new TfidfCompare());
38
39 outputFilename = "data\\" + String.format("%03d", j + 1) + "tfidf.txt";
    // 出力ファイル名を設定
40 // tf-idf の結果をファイルに保存する
41 try (FileWriter fw = new FileWriter(outputFilename)) {
42     fw.write("単語\t出現回数\tTF\tIDF\ttfidf\n");
43     for (TfidfCount tfidfCount : tfidfList) {
44         // 単語 出現回数 TF IDF tfidf
45         fw.write(tfidfCount.getWord().getHyouSoukei() + "\t"
46             + tfidfCount.getCount() + "\t"
47             + String.format("%.10f", tfidfCount.getTf()) + "\t"
48             + String.format("%.10f", tfidfCount.getIdf()) + "\t"
49             + String.format("%.10f", tfidfCount.getTfidf()) + "\n");
50     }
51     fw.close();
52 } catch (IOException ex) {
53     ex.printStackTrace();
54 }
55 tfidfList.clear(); // 次のTF-IDF計算のためにリストをクリア
56 j++; // 次のファイルのインデックスに進む
57 }
58 }
59
60 }

```

TfIdfCompare.java

Listing 15: TfIdfCompare.java

```
1 package nlp;
2
3 public class TfIdfCompare implements java.util.Comparator<TfIdfCount> {
4     @Override
5     public int compare(TfIdfCount wc1, TfIdfCount wc2) {
6         if (wc1.getTfIdf() < wc2.getTfIdf())
7             return 1;
8         if (wc1.getTfIdf() == wc2.getTfIdf())
9             return 0;
10        if (wc1.getTfIdf() > wc2.getTfIdf())
11            return -1;
12        return 0;
13    }
14
15 }
```

TfIdfCount.java

Listing 16: TfIdfCount.java

```
1 package nlp;
2
3 public class TfIdfCount extends WordCount {
4     private Double tfIdf;
5     private Double tf;
6     private Double idf;
7
8     TfIdfCount(Word word, Integer count, Double tf, Double idf, Double tfIdf) {
9         super(word, count);
10        this.tfIdf = tfIdf;
11        this.tf = tf;
12        this.idf = idf;
13    }
14
15    TfIdfCount(Word word, Integer count) {
16        super(word, count);
17        this.tfIdf = 0.0;
18    }
19 }
```

```

20     void setTfIdf(Double tfIdf) {
21         this.tfIdf = tfIdf;
22     }
23
24     Double getTfIdf() {
25         return tfIdf;
26     }
27
28     Double getTf() {
29         return tf;
30     }
31
32     Double getIdf() {
33         return idf;
34     }
35
36     void setTf(Double tf) {
37         this.tf = tf;
38     }
39
40     void setIdf(Double idf) {
41         this.idf = idf;
42     }
43 }

```

Word.java

Listing 17: Word.java

```

1  package nlp;
2
3  class Word {
4      private String hyousoukei = null; // 表層形
5      private String hinshi = null; // 品詞
6      private String hinshi1 = null; // 品詞細分類 1
7      private String hinshi2 = null; // 品詞細分類 2
8      private String hinshi3 = null; // 品詞細分類 3
9      private String katsuyoKata = null; // 活用型
10     private String katsuyoKei = null; // 活用形
11     private String genkei = null; // 原形
12     private String yomi = null; // 読み
13     private String hatsuon = null; // 発音
14
15     public boolean equals(Object obj) {

```

```

16 // オブジェクトが null の場合、不一致
17 if (obj == null) {
18     return false;
19 }
20 // オブジェクトの型が異なる場合、不一致
21 if (!(obj instanceof Word)) {
22     return false;
23 }
24 // オブジェクトの全てのフィールドが一致している場合、一致
25 if (((((Word) obj).getHyouSoukei() == null && this.getHyouSoukei() == null)
26     || ((Word) obj).getHyouSoukei().equals(this.getHyouSoukei()))
27     && (((((Word) obj).getHinshi() == null && this.getHinshi() == null)
28     || ((Word) obj).getHinshi().equals(this.getHinshi()))
29     && (((((Word) obj).getHinshi1() == null && this.getHinshi1() == null)
30     || ((Word) obj).getHinshi1().equals(this.getHinshi1()))
31     && (((((Word) obj).getHinshi2() == null && this.getHinshi2() == null)
32     || ((Word) obj).getHinshi2().equals(this.getHinshi2()))
33     && (((((Word) obj).getHinshi3() == null && this.getHinshi3() == null)
34     || ((Word) obj).getHinshi3().equals(this.getHinshi3()))
35     && (((((Word) obj).getKatsuyoKata() == null && this.getKatsuyoKata()
36         == null)
37         || ((Word) obj).getKatsuyoKata().equals(this.getKatsuyoKata
38             ()))
39     && (((((Word) obj).getKatsuyoKei() == null && this.getKatsuyoKei() ==
40         null)
41         || ((Word) obj).getKatsuyoKei().equals(this.getKatsuyoKei()))
42     && (((((Word) obj).getGenkei() == null && this.getGenkei() == null)
43     || ((Word) obj).getGenkei().equals(this.getGenkei()))
44     && (((((Word) obj).getYomi() == null && this.getYomi() == null)
45     || ((Word) obj).getYomi().equals(this.getYomi()))
46     && (((((Word) obj).getHatsuon() == null && this.getHatsuon() == null)
47     || ((Word) obj).getHatsuon().equals(this.getHatsuon())))) {
48     return true;
49 }
50 // そのほかは不一致
51 return false;
52 }
53
54 public String getHyouSoukei() {
55     return hyousoukei;
56 }
57
58 public void setHyouSoukei(String hyousoukei) {
59     this.hyousoukei = hyousoukei;
60 }

```

```
58
59     public String getHinshi() {
60         return hinshi;
61     }
62
63     public void setHinshi(String hinshi) {
64         this.hinshi = hinshi;
65     }
66
67     public String getHinshi1() {
68         return hinshi1;
69     }
70
71     public void setHinshi1(String hinshi1) {
72         this.hinshi1 = hinshi1;
73     }
74
75     public String getHinshi2() {
76         return hinshi2;
77     }
78
79     public void setHinshi2(String hinshi2) {
80         this.hinshi2 = hinshi2;
81     }
82
83     public String getHinshi3() {
84         return hinshi3;
85     }
86
87     public void setHinshi3(String hinshi3) {
88         this.hinshi3 = hinshi3;
89     }
90
91     public String getKatsuyoKata() {
92         return katsuyoKata;
93     }
94
95     public void setKatsuyoKata(String katsuyoKata) {
96         this.katsuyoKata = katsuyoKata;
97     }
98
99     public String getKatsuyoKei() {
100         return katsuyoKei;
101     }
102
```

```

103     public void setKatsuyoKei(String katsuyoKei) {
104         this.katsuyoKei = katsuyoKei;
105     }
106
107     public String getGenkei() {
108         return genkei;
109     }
110
111     public void setGenkei(String genkei) {
112         this.genkei = genkei;
113     }
114
115     public String getYomi() {
116         return yomi;
117     }
118
119     public void setYomi(String yomi) {
120         this.yomi = yomi;
121     }
122
123     public String getHatsuon() {
124         return hatsuon;
125     }
126
127     public void setHatsuon(String hatsuon) {
128         this.hatsuon = hatsuon;
129     }
130
131     public String toString() {
132         return "Word [hyousoukei=" + hyousoukei + ", hinshi=" + hinshi + ", hinshi1="
            + hinshi1 + ", hinshi2=" + hinshi2
133             + ", hinshi3=" + hinshi3 + ", katsuyoKata=" + katsuyoKata + ",
            katsuyoKei=" + katsuyoKei + ", genkei="
134             + genkei + ", yomi=" + yomi + ", hatsuon=" + hatsuon + "];"
135     }
136 }

```

WordCompare.java

Listing 18: WordCompare.java

```

1  package nlp;
2
3  import java.util.Comparator;

```



```

4
5 class WordCompare implements Comparator<WordCount> {
6     @Override
7     public int compare(WordCount wc1, WordCount wc2) {
8         if (wc1.getCount() < wc2.getCount())
9             return 1;
10        if (wc1.getCount() == wc2.getCount())
11            return 0;
12        if (wc1.getCount() > wc2.getCount())
13            return -1;
14        return 0;
15    }
16 }

```

WordCount.java

Listing 19: WordCount.java

```

1 package nlp;
2
3 class WordCount {
4     private Word word; // 形態素解析した語の結果を格納する
5     private Integer count; // 出現回数を計数するためのカウンタ
6
7     WordCount(Word word, Integer count) {
8         this.word = word;
9         this.count = count;
10    }
11
12    Word getWord() {
13        return word;
14    }
15
16    void setWord(Word word) {
17        this.word = word;
18    }
19
20    Integer getCount() {
21        return count;
22    }
23
24    void setCount(Integer count) {
25        this.count = count;
26    }

```

